UNIVERSIDAD CAECE

Cámara Argentina de Comercio y Servicios

LICENCIATURA EN CIENCIAS DE DATOS

ESTRUCTURAS Y ALGORITMOS DE DATOS II

GUIA DE LA UNIDAD Nro. 2

Contenidista: Maximiliano López

ÍNDICE

1- INTRODUCCIÓN 2

2- DESARROLLO 3

3- CONCLUSIONES 12

4- BIBLIOGRAFÍA DE REFERENCIA 12

FIGURAS:

Ilustración 1 Algoritmo principal y módulos 5
Ilustración 2 Verificación y Validación 8
Ilustración 3 Inspecciones y pruebas de software 9
Ilustración 4 Caja negra 10
Ilustración 5 Caja blanca 11

1- INTRODUCCIÓN

En la Unidad anterior, usted aprendió a medir la eficiencia de un algoritmo por medio de la Complejidad algorítmica. Además, estudió distintas funciones para representar ese comportamiento. Ahora le presentaremos una estrategia para poder construir programas más complejos y luego poder verificarlos. Para eso utilizaremos la división por módulos la cual nos permitirá dividir el problema en subproblemas y, de esa manera, simplificar el desarrollo.

2- DESARROLLO

1.1 ESTRATEGIA PARA RESOLVER PROBLEMAS

La estrategia que planteamos es pensar en el problema general e ir descomponiéndolo en subproblemas. A su vez, estos subproblemas se podrán seguir dividiendo hasta llegar a un subproblema lo bastante simple como para poder resolverse de forma sencilla.

Así es como realizaremos un algoritmo principal y subalgoritmos para ir resolviendo cada uno de los subproblemas.

Estos subalgoritmos, en programación, tienen distintos nombres, pero representan el mismo concepto. Es por eso por lo que en la bibliografía podrá encontrar los nombres de módulos, subprogramas, rutinas, subrutinas, son todos sinónimos para definir el mismo concepto de “subproblema”.

1.1.1 RAZONES PARA DEFINIR RUTINAS

Ya vimos que una de las razones para definir rutinas es que resolver un problema más simple es mucho más fácil que resolver un problema complejo. Ahora bien, hay otros motivos por los cuales debemos definir rutinas al desarrollar un algoritmo. Veamos los principales:

- **Simplifica la programación**. Una rutina es simple de programar.
- **Permite la reutilización de código**. La reutilización de código es una característica fundamental de la programación. Una vez que se programa un módulo, si ese mismo módulo hace falta para resolver otro problema, se puede utilizar sin necesidad de pensar nuevamente cómo resolverlo. Por ejemplo, si

usted realizó un módulo para calcular una raíz cuadrada, ese módulo lo podrá reutilizar en cualquier algoritmo que tenga que calcular una raíz cuadrada. Es un subproblema que ya resolvió en otro momento y no tiene que pensar nuevamente en él, porque ya sabe que está resuelto. De esta manera evitamos lo que llamamos comúnmente “reinventar la rueda”.

- **El algoritmo resulta más simple y claro**. En lugar de tener una lista muy larga de instrucciones que puede llegar a ser ilegible, el algoritmo principal será una pequeña lista de llamadas a módulos. De esta manera, al leer el algoritmo principal, se entenderá la idea de lo que realiza, sin necesidad de acceder a los detalles de cómo lo resuelve.
- **El programa es más simple para verificar**. Las pruebas se pueden realizar en cada módulo de forma independiente, en lugar de tener que probar todo el programa a la vez.
- **Reduce los tiempos de mantenimiento**. Realizar modificaciones en un programa es más simple y rápido porque dividir en módulos aísla los cambios. Una vez que se detecta en dónde realizar el cambio se deberá modificar ese módulo solamente.

1.1.2 ABSTRACCIÓN

Veremos ahora un concepto fundamental en el desarrollo de programas que se deriva de la técnica de división por módulos. Este concepto es la Abstracción.

Concretamente, la abstracción se produce cuando creamos módulos. Por ejemplo; si tenemos un algoritmo principal que se encuentra dividido en tres módulos de la siguiente manera:

```
1  Algoritmo Principal
2      Obtener Notas de Un alumno
3      Calcular Promedio De Notas
4      Mostrar Promedio de Notas
5  Fin AlgoritmoPrincipal
```

Ilustración 1 Algoritmo principal y módulos

Acá estamos aprovechando una de las ventajas de la división en módulos. Como le comentamos previamente, el algoritmo es más claro ya que, con los nombres que hemos dado a los módulos, queda bien especificado qué es lo que hace cada uno de ellos.

Analicemos qué sucede con el Módulo “Calcular Promedio De Notas”. Seguramente estará pensando que este módulo calcula el promedio de unas notas, pero no se ocupará ni de pedir las notas ni de mostrar los resultados. Este módulo está abstraído del resto, solo se ocupará de calcular el promedio. No sabemos tampoco cómo lo resolverá, ya que de eso nos ocuparemos cuando pasemos a desarrollar este módulo. Es la idea de una “caja negra”, sabemos qué necesita y qué hace el módulo, pero no sabemos cómo lo realiza.

Los módulos son independientes entre sí, aunque algunos pueden necesitar colaborar con otros, o trabajar de forma conjunta. Pero lo importante en este momento, para entender el concepto de abstracción, es comprender que cada módulo es independiente de los demás módulos y que es ideal que realice una sola tarea.

Podemos definir la abstracción como el aislamiento de un elemento de su contexto o del resto de los elementos que lo acompañan. En programación la abstracción está relacionada con “qué hace”.

1.1.3 OCULTAMIENTO DE LA INFORMACIÓN

Un concepto que se deriva de la abstracción es el de Ocultamiento de la información. Sigamos con el mismo ejemplo de la sección 1.1.2. Para calcular el promedio seguramente estará deduciendo que se realizarán operaciones intermedias, por ejemplo, sumar todos los números, contar la cantidad de números, etc. Esa información forma parte de la manera en que el módulo va a resolver su problema, no es de interés para el resto de los módulos, por eso queda “oculta” para el resto del sistema.

Continuando con el concepto de “caja negra” un módulo resuelve un único problema dejando “oculto” el cómo lo realiza.

El ocultamiento de la información también se realiza cuando hacemos el análisis previo del problema, donde definimos qué datos necesita el programa y qué resultados va a generar, sin especificar cómo se resolverá.

1.1.4 DIVISIÓN EN MÓDULOS

Ahora bien, ¿Cómo será la manera de separar el problema en módulos?

Una estrategia de desarrollo podría ser lo que se conoce como Análisis descendente (top - down).

Debemos comenzar analizando el problema a resolver, como bien lo hicimos cuando planteamos al Análisis Previo.

Luego, cuando pasamos a desarrollar el algoritmo, debemos plantear el algoritmo principal como una lista de acciones genéricas, dividiendo lógicamente el programa en acciones para **Entrada de datos, Proceso y Salida de Datos**.

¿Recuerda que esas eran las etapas de todo programa?

Esa separación en etapas nos servirá para dividir en módulos el problema.

Finalmente, debemos analizar cada uno de esos módulos y hacer el mismo proceso, es decir, dividirlos en submódulos hasta llegar a una funcionalidad mínima, en donde cada módulo realice una sola tarea.

Este proceso nos permitirá ir descendiendo en la complejidad para tener módulos, o lo que luego serán las rutinas, cada vez más simples.

1.2 VERIFICACIÓN Y VALIDACIÓN

Hasta este momento le hemos explicado todo lo relacionado con las etapas de Análisis y Desarrollo de programas. Ahora bien, una vez que usted finalice el algoritmo, seguramente la primera pregunta que se realizará es si lo que acaba de desarrollar está bien o tiene algún error.

Este es el momento de realizar la prueba del algoritmo. Para comenzar, le explicaremos algunas ideas para ponernos de acuerdo en los conceptos principales.

1.2.1 VERIFICACIÓN

En este proceso usted deberá comprobar que el Software cumple con los requerimientos funcionales y no funcionales, es decir si cumple la especificación. En esta etapa se responde a la pregunta ¿Se está construyendo el producto correctamente?

1.2.2 VALIDACIÓN

Este proceso es más general, aquí usted deberá comprobar que el Software cumple con las expectativas del cliente, no solo con lo que está especificado. En esta etapa se responde a la pregunta ¿Se está construyendo el producto concreto?

De esta manera, podemos ver que la Verificación y Validación son los procesos de análisis que aseguran que el software cumple con la especificación de requerimientos y las expectativas del usuario.

Ilustración 2 Verificación y Validación

1.2.3 TÉCNICAS DE VERIFICACIÓN Y VALIDACIÓN

Existen dos técnicas mediante las cuales usted podrá verificar y validar el Software, y las mismas se caracterizan dependiendo del momento en el cual se estén realizando. Así es como tenemos:

- **Inspección de Software:** Aquí se analizan diferentes representaciones del sistema en búsqueda de errores. (diagramas, documentos de análisis, código fuente). No hace falta que el programa esté desarrollado totalmente.

Este proceso es ESTÁTICO y se realiza durante TODO EL PROCESO DE DESARROLLO.

- **Pruebas de Software:** En donde se constata dinámicamente el resultado de la ejecución del SW contra lo esperable. Usted podrá definir un conjunto de datos de prueba y luego ejecutar el software ingresando como entrada esos mismos datos, los resultados obtenidos deberá contrastarlos con los que estaba esperando. Para realizar estas pruebas se necesita una versión ejecutable del programa.

Este proceso es DINÁMICO y se realiza LUEGO DEL DESARROLLO.

En la siguiente imagen le presentamos el momento en que se realizan las inspecciones y pruebas de software. Las inspecciones se realizan durante todo el proceso de desarrollo, mientras que para realizar las pruebas será necesario contar con el código fuente ejecutable o un prototipo.

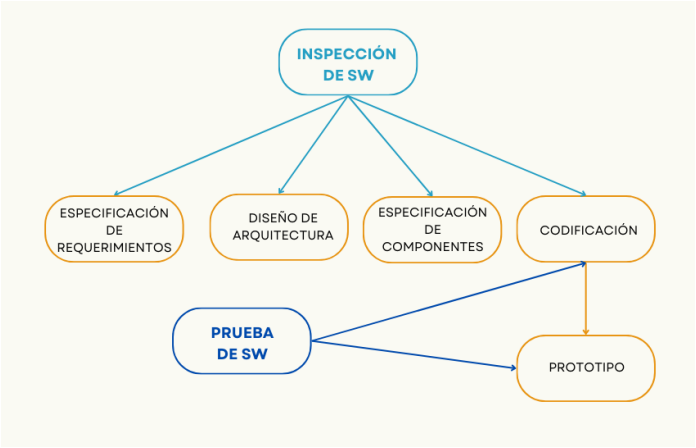

Ilustración 3 Inspecciones y pruebas de software

1.2.4 ETAPAS DEL PROCESO DE PRUEBAS

Hay muchas técnicas diferentes para realizar las pruebas de software y asegurar su calidad. A continuación, le presentamos algunas de las técnicas más comunes:

- **Prueba de unidades.** Se prueban Métodos, rutinas, clases, de manera independiente. Esto se puede realizar cuando finaliza el desarrollo de cada rutina.
- **Prueba de integración**. Se prueban agrupaciones de elementos del sistema. Cuando las rutinas están validadas luego de la prueba de unidades, se pueden ir integrando al sistema y realizar la prueba en conjunto.
- **Prueba de sistema**. Se prueba el sistema completo. Esta prueba se realiza una vez que están desarrollados todos los componentes.
- **Prueba de aceptación**. Se prueba el sistema en un entorno real con la participación del usuario.

1.2.5 CAJA NEGRA VS CAJA BLANCA

Las pruebas de caja negra y caja blanca son dos técnicas diferentes de pruebas de software. Son muy importantes en el proceso de validación de rutinas y de su integración en el software. A continuación, le presentamos estas técnicas para que pueda realizar sus evaluaciones adecuadamente.

- **Caja negra (pruebas funcionales)**: Las pruebas de caja negra se enfocan en la funcionalidad del software y se realizan sin tener en cuenta la estructura interna del código fuente. Si usted va a realizar una prueba de caja negra, no necesita conocer la lógica o el diseño interno del software. En estas pruebas, se evalúa el comportamiento del software desde el punto de vista del usuario, tratando de detectar errores o comportamientos inesperados.

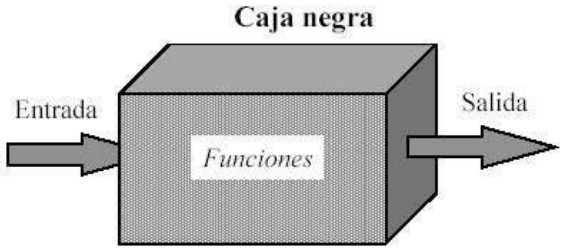

Ilustración 4 Caja negra

- **Caja blanca (pruebas estructurales)**: Estas pruebas también conocidas como pruebas estructurales, se enfocan en la estructura interna del software. Usted deberá tener acceso al código fuente del software y tener conocimiento de su arquitectura y diseño para poder realizar una prueba de caja blanca. En estas pruebas se buscan identificar errores en el código, como por ejemplo bucles infinitos, problemas de lógica, redundancias, entre otros.

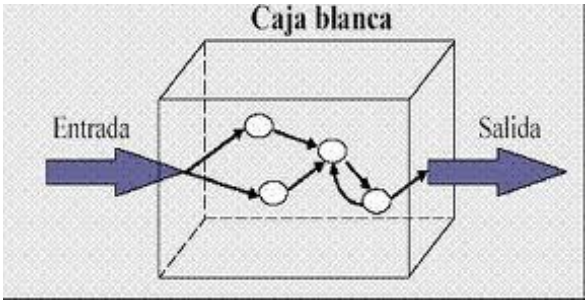

Ilustración 5 Caja blanca

A modo de resumen, le podemos comentar que las pruebas de caja negra se enfocan en la funcionalidad, comportamiento y resultados esperados del software, mientras que las pruebas de caja blanca se enfocan en la estructura interna del software y en el código fuente para identificar posibles errores y mejoras en el diseño.

Ambas técnicas son importantes en el proceso de pruebas de software y se complementan entre sí para asegurar la calidad y confiabilidad del software.

3- CONCLUSIONES

En esta unidad le presentamos las técnicas para VALIDACIÓN DE PROGRAMAS. Para eso comenzamos con un repaso sobre la técnica de DIVISIÓN EN MÓDULOS, como una estrategia para resolver problemas. Luego, le comentamos los conceptos de VERIFICACIÓN y VALIDACIÓN, mediante los cuales podrá evaluar si se cumplen los requerimientos funcionales y no funcionales del sistema, junto con las expectativas del cliente. Finalmente, como técnicas diferentes de pruebas de software le presentamos las pruebas funcionales y estructurales, denominadas habitualmente CAJA NEGRA y CAJA BLANCA.

4- BIBLIOGRAFÍA DE REFERENCIA

Bisbal Riera, J. (2013). Manual de algorítmica: recursividad, complejidad y diseño de algoritmos. Editorial UOC. https://elibro.net/es/ereader/caece/56561?page=50

Sedgewick, R. y Wayne, K. (2011). Algorithms. 4th Edition. Addison- Wesley.

Cormen, T., Leiserson, C., Rivest, R. & Stein, C. (2022) Introduction to algorithms. 4th Edition. The MIT Press.